Assignment 5: Perceptron

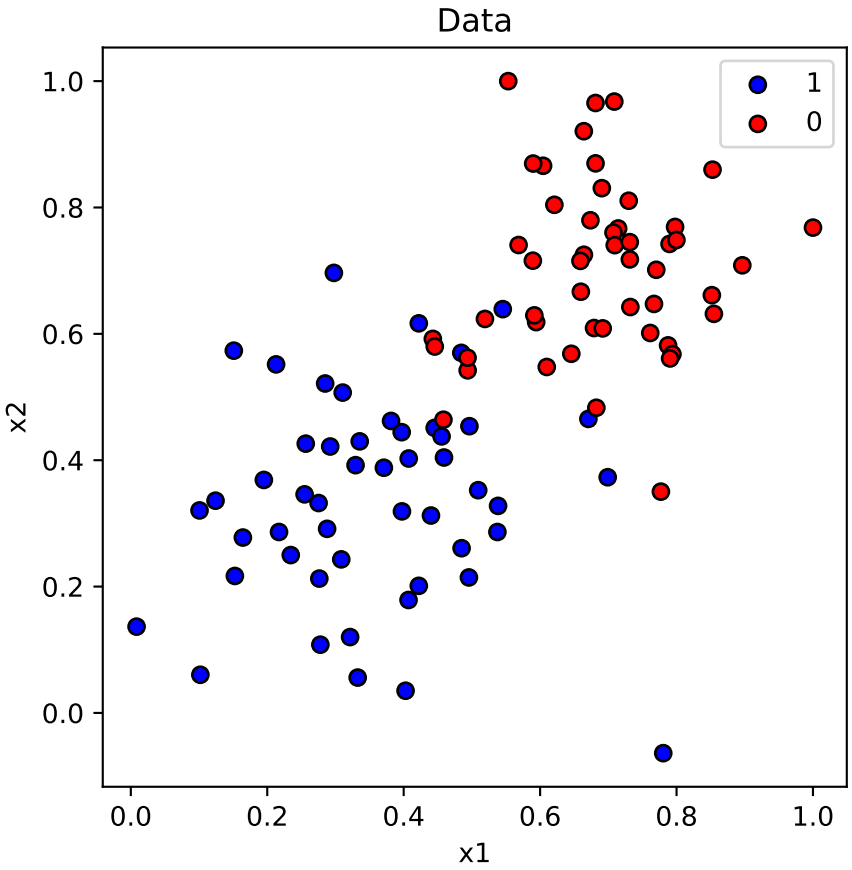
Rowan Morkner

2026-05-15

Data

100 points, 50/50 split between two classes. Both features sit in roughly [0, 1]. Blues (label=1) cluster in the lower-left, reds (label=0) in the upper-right, with a chunky overlap band along the diagonal. Looks linearly separable but not perfectly — a few stragglers will always end up on the wrong side.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv('data.csv', header=None, names=['x1', 'x2', 'y'])
X = df[['x1', 'x2']].values
y = df['y'].values

plt.figure(figsize=(5, 5))
plt.scatter(X[y==1, 0], X[y==1, 1], c='blue', edgecolor='k', label='1')
plt.scatter(X[y==0, 0], X[y==0, 1], c='red', edgecolor='k', label='0')
plt.xlabel('x1'); plt.ylabel('x2'); plt.legend(); plt.title('Data')
plt.show()
```



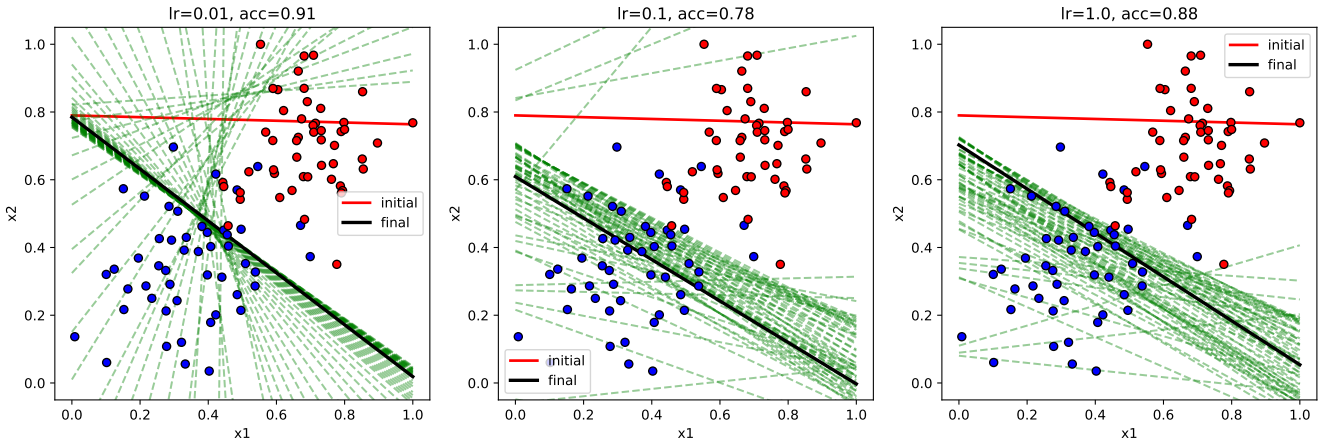
Part 1: Heuristic Perceptron

Step-function classifier. For each misclassified point, push the line in the direction of the correct label by a full learning-rate step. No notion of "how wrong" — every miss gets the same nudge.

```
def step(z):
    return (z >= 0).astype(int)

def line_from_weights(w, b, x_range):
    return -(w[0] * x_range + b) / w[1]

def heuristic_perceptron(X, y, lr, n_iter, seed=1):
    rng = np.random.default_rng(seed)
    w = rng.uniform(-1, 1, size=2)
    b = rng.uniform(-1, 1)
    history = [(w.copy(), b)]
    for _ in range(n_iter):
        for xi, yi in zip(X, y):
            pred = step(np.dot(w, xi) + b)
            if pred != yi:
                if yi == 1:
                    w += lr * xi
                    b += lr
                else:
```

```
                    w -= lr * xi
                    b -= lr
        history.append((w.copy(), b))
    return w, b, history

def plot_history(X, y, history, title):
    x_range = np.array([0, 1])
    plt.figure(figsize=(5, 5))
    plt.scatter(X[y==1, 0], X[y==1, 1], c='blue', edgecolor='k', zorder=3)
    plt.scatter(X[y==0, 0], X[y==0, 1], c='red', edgecolor='k', zorder=3)
    w0, b0 = history[0]
    plt.plot(x_range, line_from_weights(w0, b0, x_range), 'r-', linewidth=2, label='initial')
    for w, b in history[1:-1]:
        plt.plot(x_range, line_from_weights(w, b, x_range), 'g--', alpha=0.4)
    wf, bf = history[-1]
    plt.plot(x_range, line_from_weights(wf, bf, x_range), 'k-', linewidth=2.5, label='final')
    plt.xlim(-0.05, 1.05); plt.ylim(-0.05, 1.05)
    plt.xlabel('x1'); plt.ylabel('x2'); plt.title(title); plt.legend()
    plt.show()

def accuracy(X, y, w, b):
    return (step(X @ w + b) == y).mean()
```

Learning rate sweep



lr	accuracy
0.01	0.91
0.10	0.78
1.00	0.88

The non-monotonic result is the interesting bit: lr=0.01 beats both larger rates. The reason is that the heuristic update has no concept of confidence — every misclassified point shoves the line by the full learning rate regardless of how close it was to the boundary. At lr=0.1, points along the overlap band keep yanking the line back and forth and it never settles into a good orientation, ending at 0.78. At lr=0.01 those same shoves are tiny so the line drifts

slowly and ends up in a stable spot near the gap between clusters. lr=1.0 is wildly noisy (you can see the green fan spreading everywhere) but happens to land somewhere reasonable.

The lesson: with the heuristic approach, "convergence" is more about luck and step size than about iteration count. There's no loss to monitor and no guarantee that a smaller step would help — it depends on the data.

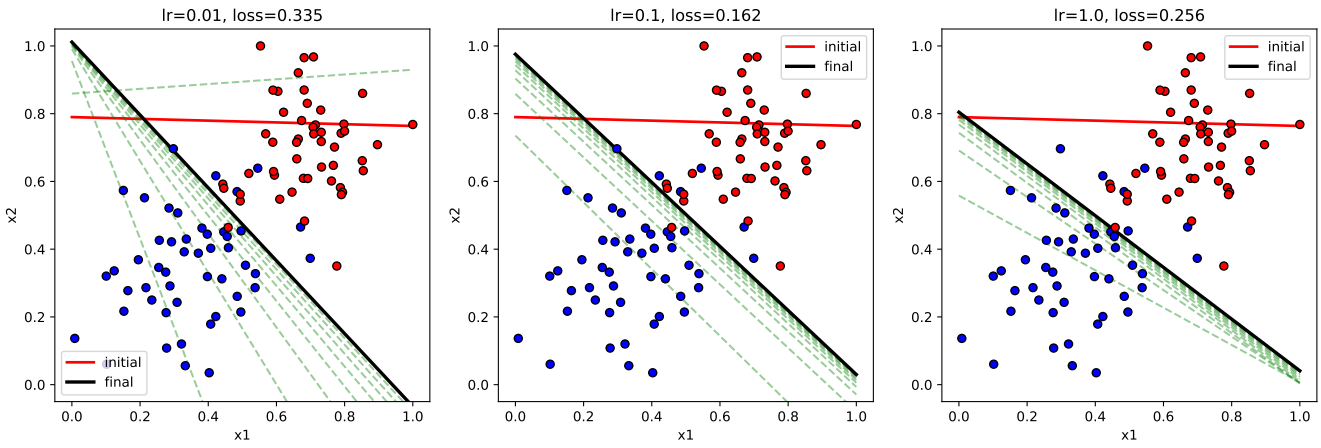
Part 2: Gradient Descent Perceptron

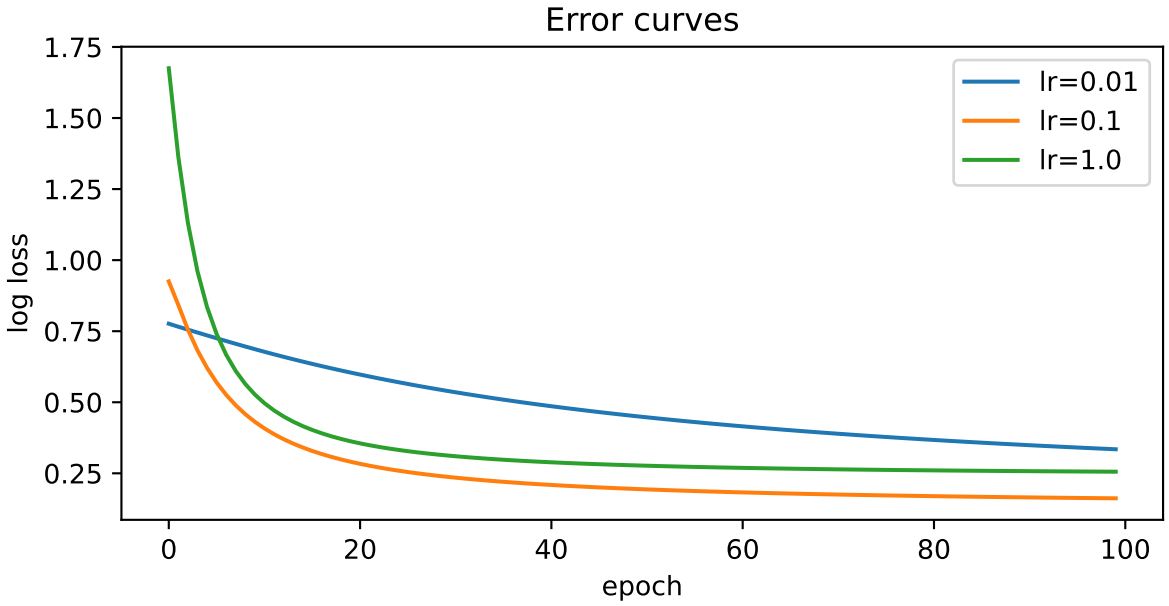
Sigmoid output → continuous probability. The error term $y - \hat{y}$ is small for confident-correct predictions and large for confident-wrong ones, so updates naturally scale with how wrong we are. We can also actually compute a loss.

```
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def log_loss(y, y_hat):
    eps = 1e-12
    y_hat = np.clip(y_hat, eps, 1 - eps)
    return -np.mean(y * np.log(y_hat) + (1 - y) * np.log(1 - y_hat))

def gd_perceptron(X, y, lr, n_epochs, seed=1):
    rng = np.random.default_rng(seed)
    w = rng.uniform(-1, 1, size=2)
    b = rng.uniform(-1, 1)
    history = [(w.copy(), b)]
    errors = []
    for epoch in range(n_epochs):
        for xi, yi in zip(X, y):
            y_hat = sigmoid(np.dot(w, xi) + b)
            err = yi - y_hat
            w += lr * err * xi
            b += lr * err
        errors.append(log_loss(y, sigmoid(X @ w + b)))
        if epoch % 10 == 0:
            history.append((w.copy(), b))
    history.append((w.copy(), b))
    return w, b, history, errors
```

Learning rate sweep (100 epochs)

```
plt.figure(figsize=(7, 3.2))
plt.plot(errs01, label='lr=0.01')
plt.plot(errs1, label='lr=0.1')
plt.plot(errs10, label='lr=1.0')
plt.xlabel('epoch'); plt.ylabel('log loss'); plt.title('Error curves')
plt.legend(); plt.show()
```



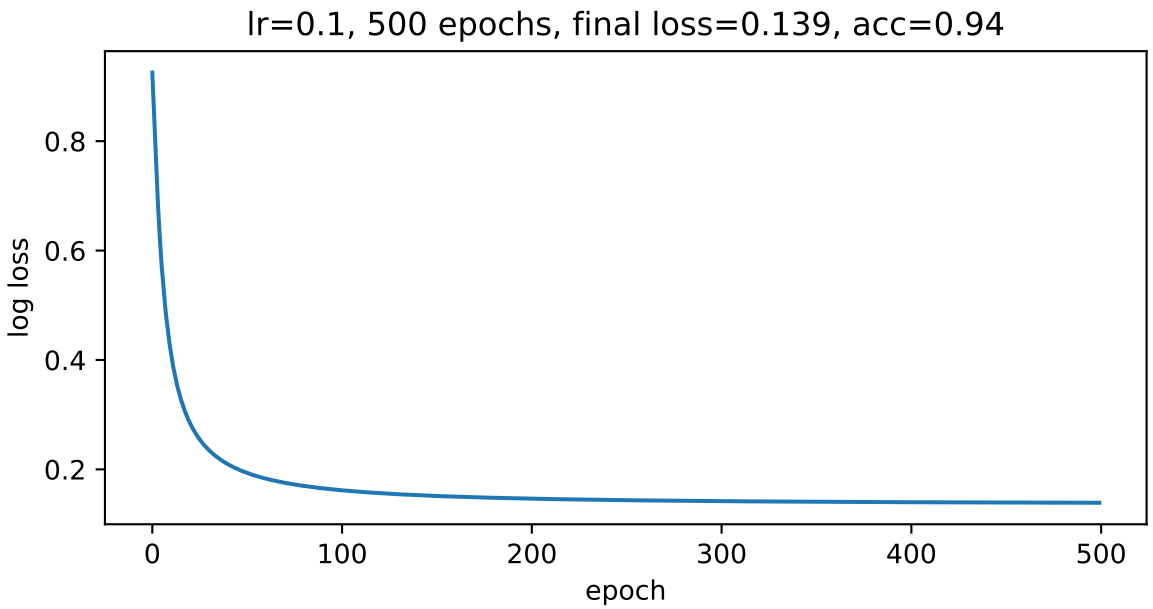


lr	final log loss	accuracy
0.01	0.335	0.93
0.10	0.162	0.93
1.00	0.256	0.93

All three reach the same accuracy (0.93) but the loss tells a richer story. lr=0.01 is too small — the curve is still dropping at epoch 100 and final loss is double the best. lr=0.1 is the sweet spot, monotonic and smooth. lr=1.0 drops fast but the curve has visible bumps from overshooting; final loss is worse than lr=0.1 because the big steps keep flying past the minimum.

Pushing epochs

```
w, b, hist, errs_long = gd_perceptron(X, y, lr=0.1, n_epochs=500)
plt.figure(figsize=(7, 3.2))
plt.plot(errs_long)
plt.xlabel('epoch'); plt.ylabel('log loss')
plt.title(f'lr=0.1, 500 epochs, final loss={errs_long[-1]:.3f}, acc={accuracy(X,y,w,b):.2f}')
plt.show()
```



500 epochs at lr=0.1 pushes loss to 0.14 and accuracy to 0.94. The curve is flat by epoch ~150 — more epochs don't help. There's a floor at around 0.14 because the overlap band has points that genuinely sit on the wrong side and no straight line can fix that.

Conclusion

Both methods land at roughly the same boundary, but gradient descent is much more reliable and informative. The heuristic perceptron has no notion of confidence so learning rate choice is unpredictable (lr=0.01 outperformed lr=0.1 here, which is surprising and not something you'd guess from the data). The GD version's sigmoid scales updates by confidence, converges smoothly, and gives you a loss curve to actually monitor — which is the real practical advantage.

For this dataset: lr=0.1 with 100+ epochs of gradient descent gets you 93–94% accuracy, and that's basically the ceiling given the class overlap.